

7.1)

Corrected comments:

```
// Calculates the greatest common divisor of two
// numbers using the Euclidean algorithm
private long GCD( long a, long b )
{
    // loop until b is a divisor of a
    for( ; ; )
    {
        long remainder = a % b;
        // When the remainder is zero, b is the GCD.
        If( remainder == 0 ) return b;
        // update a and b for the next loop
        a = b;
        b = remainder;
    };
}
```

7.2)

The programmer may have used a top-down design, pseudo-coding the function with high-level comments and leaving detailed descriptions of code statements in the comments.

The second way could have been if the programmer added the comments after writing the code to help make the code more human-readable.

7.4)

To make the code offensive, you can validate the inputs and the result as soon as possible, rather than letting incorrect values propagate. One way to do this is to use a `Debug.Assert` to check that a and b never reach 0 and throw an error if it does.

7.5)

No, the offensive coding will catch problematic values and throw an error, but that error should be caught and handled by the code calling the GCD function, not inside the function itself.

7.7)

Highest-Level Instructions

1. Locate the car
2. Get into the car
3. Start the car and put it into drive
4. Type supermarket into the car's GPS
5. Drive along the route suggested by the GPS
6. Park the car at the supermarket.

Assumptions

- The car is functional and has fuel

- The driver is licensed to operate the car
- The GPS knows which supermarket is closest
- Roads to the supermarket are accessible and legal to drive on
- Parking at the supermarket is available

8.1)

Pseudo-code

// Very inefficient function to check and validate if two numbers are relatively prime

function BruteForceRelativelyPrime(a, b):

 a = abs(a)

 b = abs(b)

 for i from 2 to min(a, b):

 if a % i == 0 AND b % i == 0:

 return false

 return true

// function that checks if the brute force approach and efficient approach return the same value

function TestIsRelativelyPrime():

 for a from -1000000 to 1000000:

 for b from -1000000 to 1000000:

 expected = BruteForceRelativelyPrime(a, b)

 result = IsRelativelyPrime(a, b)

 if result != expected:

 print("Error with values:", a, b)

8.3)

The brute force method I used is an exhaustive testing approach since it checks all possible values to see if any of them break the relative prime-ness of the numbers.

The test method then uses that result to black box test the efficient function against the base trueness from the brute force method.

8.5)

Functions written in Python:

def gcd(a, b):

 """Compute the greatest common divisor using Euclid's algorithm."""

 a = abs(a)

 b = abs(b)

 # If either value is 0, return the other

```

if a == 0:
    return b
if b == 0:
    return a

while True:
    remainder = a % b
    if remainder == 0:
        return b
    a = b
    b = remainder

```

translated from C#

```

def are_relatively_prime(a, b):
    """Return True if a and b are relatively prime."""
    # Only 1 and -1 are relatively prime to 0
    if a == 0:
        return b == 1 or b == -1
    if b == 0:
        return a == 1 or a == -1

    g = gcd(a, b)
    return g == 1 or g == -1

```

```

def brute_force_relatively_prime(a, b):
    """Simple but slower method used for testing."""
    a = abs(a)
    b = abs(b)

    if a == 0:
        return b == 1
    if b == 0:
        return a == 1

    for i in range(2, min(a, b) + 1):
        if a % i == 0 and b % i == 0:
            return False
    return True

```

```

def test_are_relatively_prime():
    """Test the function across a range of values."""
    errors = 0

```

```

for a in range(-100, 101):
    for b in range(-100, 101):
        expected = brute_force_relatively_prime(a, b)
        result = are_relatively_prime(a, b)

        if expected != result:
            print(f"Error for ({a}, {b}) expected {expected} got {result}")
            errors += 1

if errors == 0:
    print("All tests passed!")
else:
    print(f"{errors} errors found.")

```

```

# Run the tests
test_are_relatively_prime()

```

I didn't find any bugs, but since my method uses `abs()`, the `is_relatively_prime()` function doesn't need to check if the factors are 1 or -1, but can just check 1 now.

8.9)

Exhaustive testing is black-box testing because it:

- only checks inputs and outputs
- does not depend on internal code implementation

8.11)

Bug sets:

Alice: {1,2,3,4,5} → 5

Bob: {2,5,6,7} → 4

Carmen: {1,2,8,9,10} → 5

Lincoln estimate:

Alice/Bob: $5 \times 4 \div 2 = 10$

Alice/Carmen: $5 \times 5 \div 2 = 12.5$

Bob/Carmen: $4 \times 5 \div 1 = 20$

Unique bugs found:

{1,2,3,4,5,6,7,8,9,10} = 10

Most bugs estimated vs unique bugs found

$20 - 10 = 10$

Remaining ≈ 10

Meaning testers may have only found half of the bugs in the worst-case scenario.

8.12)

Lincoln formula:

$$\text{Total} = (A \times B) / \text{overlap}$$

If overlap = 0:

division by zero → infinite estimate

Meaning:

The estimate becomes unreliable

Testers may be finding different areas of the system

Lower bound estimate:

At minimum:

$$A + B$$

Because those bugs definitely exist.

Example:

If testers found 10 and 8 bugs:

$$\text{minimum bugs} \geq 18$$